

boltzmann

October 5, 2026



UNIVERSIDAD DE COLIMA

Facultad de Ingeniería Mecánica y Eléctrica

Ingeniería en Computación Inteligente

7mo B

Integrantes del equipo:

Christian David Sánchez Sánchez

Marcela Betzabeth Márquez González

Fecha de entrega: 5 de octubre de 2026

0.1 Introducción

Una máquina de Boltzmann es un tipo específico de red neuronal estocástica cuya estructura incluye unidades de procesamiento interconectadas que operan de manera probabilística. A diferencia de otras redes neuronales, las máquinas de Boltzmann son especialmente útiles para modelar y simular sistemas complejos y dinámicos debido a que pueden capturar la variabilidad e incertidumbre inherentes en muchos problemas del mundo real [1]. Una de sus características principales es que se considera una red neuronal *recurrente*, las cuales se distinguen por su “memoria” ya que toman información de entradas anteriores para influir en la entrada y salida actual [2].

¿Cómo puede una red neuronal estocástica aprender una distribución de probabilidad y cuáles son las dificultades que limitan su entrenamiento? Para poder responder a esta pregunta, es necesario primero entender los conceptos básicos que nos llevan a cuestionarla. Un *proceso estocástico* es todo proceso que describe la evolución de un sistema a lo largo del tiempo donde el azar tiene una intervención. A diferencia de los modelos deterministas, donde el resultado está totalmente definido por las condiciones iniciales, en estos procesos hay incertidumbre en los resultados [3].

De este modo, podemos definir a las *redes neuronales estocásticas* como modelos caracterizados por las variaciones aleatorias ya sea en sus neuronas o en sus pesos. Estas resultan especialmente útiles para resolver problemas de optimización ya que, gracias a estas fluctuaciones, podemos crear algoritmos que eviten los mínimos locales además de modelar y simular sistemas complejos.

Su principal limitación son los tiempos de entrenamiento, ya que es extremadamente lento y computacionalmente caro debido a la necesidad de calcular el equilibrio térmico mediante métodos de muestreo avanzados.

0.2 Antecedentes

En 1982, John Hopfield introdujo la *Red de Hopfield*, una red neuronal recurrente de unidades binarias con conexiones simétricas que funcionaba como un sistema dinámico con memoria asociativa. Su funcionamiento es simple: dada una configuración inicial, la red actualiza asincrónicamente el estado de sus nodos hasta lograr la convergencia a un estado estable. Este modelo, a pesar de haber revolucionado el área de la inteligencia artificial, su dinámica descendente provocaba que la red se quedara bloqueada en configuraciones no óptimas sin poder alcanzar un mínimo global [4].

Para resolver las limitaciones en sistemas energéticos, se desarrolló la técnica de *Recocido Simulado*, el cual introduce ruido estocástico controlado en las reglas de transición entre estados para que el sistema adquiera la capacidad de realizar saltos probabilísticos hacia estados de mayor energía. Cuando se obtienen temperaturas elevadas, el sistema explora el espacio disponible de estados; a medida que la temperatura disminuye, su probabilidad de visitar estados desfavorables también [5].

En 1985 se consolidó la *máquina clásica de Boltzmann*, unificando la estructura de la *red de Hopfield* y la técnica del *Recocido Simulado*. Entre sus tantas innovaciones y contribuciones teóricas están [2]:

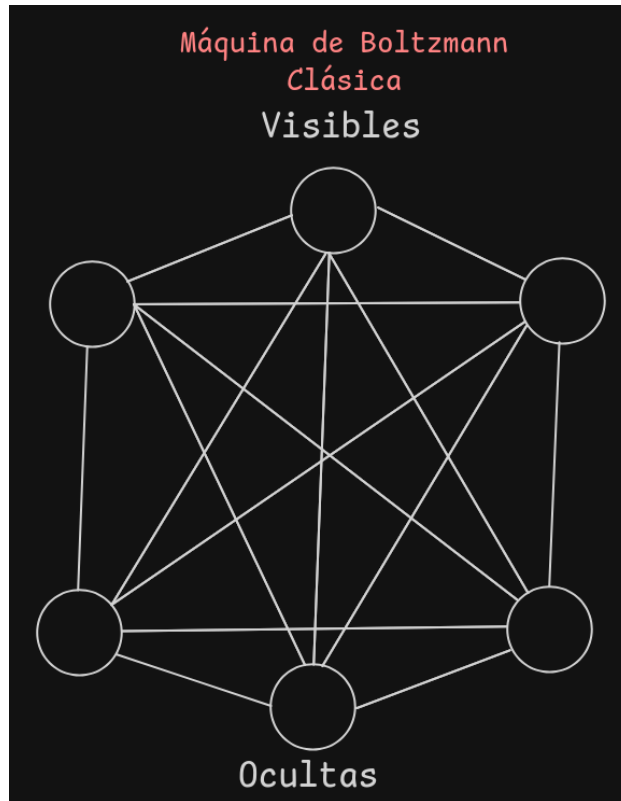
- Se dividieron las neuronas en unidades visibles (las cuales reciben la entrada/salida de datos) y unidades ocultas (las cuales capturan correlaciones complejas).
- Se reemplazó la función de activación determinista por reglas de transición probabilística.
- Se demostró que, al correr la red libremente a una temperatura fija durante el tiempo sufi-

ciente, la probabilidad de observar cualquier estado global converge a la distribución estocástica de Boltzmann.

A pesar de tener un sustento y solidez matemática, este modelo resulta computacionalmente inviable para aplicaciones prácticas debido al intenso costo computacional que representa utilizarlo. Se propusieron algunos modelos derivados que superan este obstáculo, como la *Máquina de Boltzmann Restringida* o la *Restricción Bipartita* [6].

0.3 Arquitectura

La arquitectura de una máquina de Boltzmann se puede describir con el siguiente diagrama:



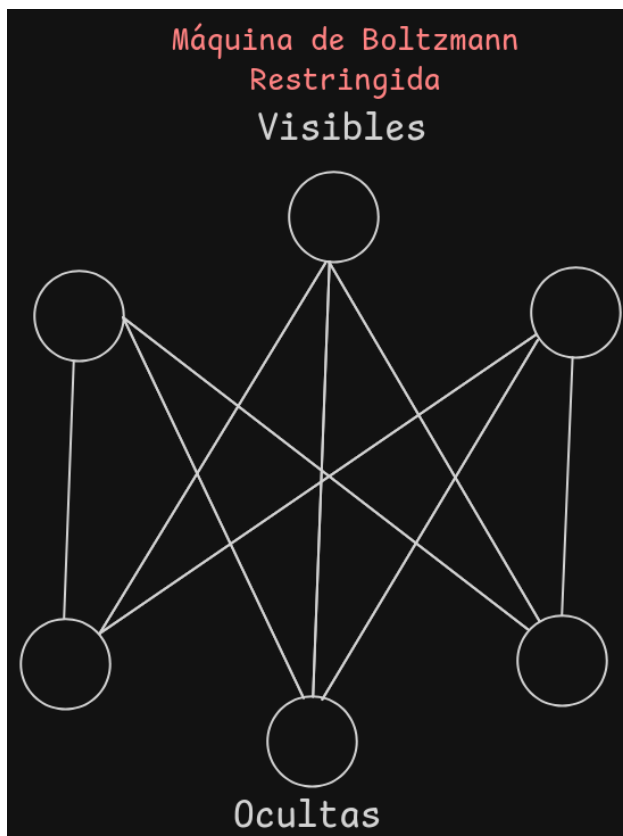
En donde:

- **Unidades visibles:** Representan las variables observables del entorno (datos de entrada) [7].
- **Unidades ocultas:** Actúan como detectores de características no lineales que capturan correlaciones complejas entre variables visibles [7].
- **Pesos (W):** Representa el peso de la conexión simétrica entre la unidad visible y la unidad oculta, es decir, que tan importante será esa entrada en la toma de decisiones [7].
- **Sesgos:** Cada unidad visible/oculta tiene un valor de sesgo, el cual acciona a cada unidad para que ejecute una acción y nunca quede sin hacer nada (muerta) [7].

Cada una de estas unidades tiene dos estados posibles: 0 o 1. Es decir, sus estados son totalmente binarios.

Como podemos notar en esta arquitectura, la clave es que cada una de las neuronas está conectada entre si, incluso las de la misma capa, lo que hace que cada unidad sea dependiente de otras las cuales provienen de multiples capas.

En cambio, la arquitectura de la máquina de Boltzmann restringida es:



Podemos notar que la principal diferencia entre ambas es que esta (RBM) prohíbe explícitamente las conexiones internas dentro de la misma capa. Esto garantiza que, dado el estado de la capa visible todas las unidades ocultas y visibles son condicionalmente independientes entre si por lo que se permite realizar inferencia probabilística [8].

0.4 Comportamiento probabilístico

El estado de las unidades en arquitecturas deterministas (como la red de Hopfield o perceptrón) se rige por la función escalón o umbral, la cual indica que si una neurona se activa basandose en el valor de la suma ponderada. El uso de esta función es bastante simple y entendible, pero tiene un problema en modelos basados en energía: no permite transicionar a estados de mayor energía, lo que ocasiona que pueda quedar atrapado en mínimos locales. Además, elimina el factor estocástico el cual es necesario para permitir al modelo atravesar barreras energéticas, escapar de mínimos locales o explorar el espacio de estados a su conveniencia [1].

La expresión utilizada es la siguiente:

$$P(s_i = 1) = \frac{1}{1 + e^{-\frac{\Delta E_i}{T}}}$$

Donde:

- $P(s_i = 1)$: Representa la probabilidad de que la neurona cambie o se mantenga en el estado activo, dado el estado del resto de las neuronas presentes en la red.
- s_i : Variable estocástica binaria que representa su estado actual (0 o 1).
- ΔE_i : Representa la diferencia de energía o la entrada neta, siendo la variación global que experimenta la red cuando alguna neurona se activa.
- T : Representa el nivel de ruido estocástico o fluctuación térmica del sistema. En caso de que $T = 0$, esta actuará como una función escalón convencional, conforme aumenta su valor el ruido que introduce también lo hace y cuando disminuye el ruido que introduce también lo hace.
- e : Constante de Euler, la cual es la base de los logaritmos naturales.
- $\frac{1}{1+e^{-\frac{\Delta E_i}{T}}}$: Representa la función logística la cual toma cualquier valor de entrada y lo mapea a un rango continuo entre 0 y 1, lo que nos permite interpretarlo como una probabilidad.

Al realizar dos ejecuciones, el sistema produce dos secuencias de estados diferentes debido a la naturaleza estocástica inherente al modelo y a su mecanismo de transiciones probabilísticas. El estado global de la red (ΔE_i) se actualiza paso a paso mediante el *Muestreo de Gibbs*, realizando un proceso de “cadena” donde cada estado x_{i+1} se muestrea probabilísticamente con todos sus estados anteriores.

0.5 Función de energía

Para definir estados binarios como energía, se utiliza la siguiente ecuación:

$$E(s) = -\frac{1}{2} \sum_i \sum_j w_{ij} s_i s_j - \sum_i b_i s_i$$

Esta ecuación define como calcular la energía global del estado ($E(s)$) basado en los siguientes parámetros:

- $E(s)$: Valor escalar que mide el nivel de conflicto o desacuerdo en la red para una configuración particular de neuronas s . A menor valor de energía, menor será la probabilidad de que el sistema visite o genere dicho estado.
- w_{ij} : Representa la intensidad de la conexión e interacción entre la neurona i y la neurona j . Entre mayor sea el valor, la red premiará la activación simultánea de ambas neuronas por lo que la energía disminuirá.
- s_i, s_j : Variables estocásticas y binarias de las neuronas i y j . El valor de su producto es máximo 1.
- Factor $\frac{1}{2}$: Al realizar una doble suma sobre todos los índices i y j , cada par de neuronas (i, j) y (j, i) se contabiliza dos veces, por lo que se debe corregir para que solo contribuya una sola vez a la energía total del sistema.
- b_i : Parámetro que determina la preferencia inherente de la neurona i a estar activa, independientemente del estado actual de las demás neuronas.
- s_i (Segundo termino): Representa el estado de la neurona i , multiplicando al sesgo b_i , lo que garantiza que el sesgo solo altere la energía global cuando la neurona esté encendida.

Ahora, ¿por qué el signo negativo? Este garantiza que las configuraciones deseables reduzcan la energía global y, por ende, tengan mayor probabilidad de ocurrencia.

Realizaremos el cálculo de 3 estados. Para efectos de la práctica, se utilizaron los siguientes valores:

- $S_1 = (1, 1, 0)$
- $S_2 = (1, 0, 1)$
- $S_3 = (0, 0, 0)$
- $W_{12} = 2$
- $W_{13} = -1.5$
- $W_{23} = 1$
- $b_1 = 0.5$
- $b_2 = -1$
- $b_3 = 0$

$W_{12} = 2$	$W_{13} = -1.5$	$W_{23} = 1$	$b_1 = 0.5$	$b_2 = -1$	$b_3 = 0$
Estado 1: $S_1 = (1, 1, 0)$	Estado 2: $S_2 = (1, 0, 1)$	Estado 3: $S_3 = (0, 0, 0)$			
$W_{12} S_1 S_2 = 2 \cdot 1 \cdot 1 = 2$	$W_{12} S_1 S_2 = 2 \cdot 1 \cdot 0 = 0$	$W_{12} S_1 S_3 = 2 \cdot 0 \cdot 0 = 0$			
$W_{13} S_1 S_3 = -1.5 \cdot 1 \cdot 0 = 0$	$W_{13} S_1 S_3 = -1.5 \cdot 1 \cdot 1 = -1.5$	$W_{13} S_1 S_3 = -1.5 \cdot 0 \cdot 0 = 0$			
$W_{23} S_2 S_3 = 1 \cdot 1 \cdot 0 = 0$	$W_{23} S_2 S_3 = 1 \cdot 0 \cdot 1 = 0$	$W_{23} S_2 S_3 = 1 \cdot 0 \cdot 0 = 0$			
Suma total = 2	Suma total = -1.5	Suma total = 0			
Sesgos:	Sesgos:	Sesgos:			
$0.5 \cdot 1 + (-1)(1) + (0)(0)$	$(0.5)(1) + (-1)(0) + (0)(1)$	$(0.5)(0) + (-1)(0) + (0)(0)$			
$= 0.5 - 1 + 0 = -0.5$	$= 0.5 + 0 = 0.5$	$= 0$			
$E(S_1) = 2 - (-0.5) = -1.5$	$E(S_2) = -1.5 - 0.5 = -1$	$E(S_3) = -0 + 0 = 0$			

Podemos notar que el estado 1 es el que resultó con la menor energía, mientras que el estado 2 fue el mayor.

El comportamiento probabilístico general de estas máquinas se basa en la conexión de la energía, estabilidad y probabilidad.

La relación entre la energía y la probabilidad en las máquinas de Boltzmann está formalizada por la distribución de Gibbs-Boltzmann, según la cual la probabilidad conjunta de que el sistema se encuentre en un estado global determinado s disminuye de manera exponencial a medida que aumenta su función de energía $E(s)$. Las configuraciones con menor energía adquieren una mayor probabilidad de ocurrencia al representar estados estables que satisfacen óptimamente las restricciones de la red, mientras que los estados de alta energía son penalizados y resultan poco probables.

0.6 Distribución de Boltzmann

La probabilidad normalizada de un estado se define por:

$$P(s) = \frac{e^{-\frac{E(s)}{T}}}{Z}$$

Esto debido a que las configuraciones de menor energía resultan en una mayor probabilidad.

Su función de partición es:

$$Z = \sum_s e^{-\frac{E(s)}{T}}$$

La función de partición Z constituye la constante de normalización de la distribución de probabilidad de Gibbs-Boltzmann, encargada de transformar los factores energéticos unificados $e^{-E(s)/T}$ en valores de probabilidad formalmente válidos. Su cálculo exige considerar la totalidad de los estados

posibles del sistema porque, de acuerdo con los fundamentos de la probabilidad, la suma de las probabilidades asignadas a cada una de las configuraciones dentro del espacio de estados debe ser exactamente igual a uno. Si Z omitiera tan solo una configuración global, la constante de normalización resultante sería inferior a la real, lo que provocaría que la suma total de las probabilidades superara la unidad y desvalidaría el modelo probabilístico. Cabe señalar además que la convención matemática asigna un signo negativo a la energía en el exponente ($P(s) = \frac{e^{-E(s)/T}}{Z}$), garantizando que los estados de menor energía posean probabilidades de ocurrencia más elevadas.

La severa dificultad computacional para evaluar Z radica en el crecimiento exponencial del espacio de estados a medida que aumenta el número de neuronas en la red. En un sistema compuesto por N unidades binarias (visibles y ocultas), cada neurona puede adoptar dos estados independientes (0 o 1), lo que genera un total de 2^N configuraciones globales posibles. Como la suma requerida para evaluar Z debe iterar sobre este universo completo de 2^N estados, el número de operaciones matemáticas requeridas crece con una complejidad temporal de $\mathcal{O}(2^N)$. Mientras que para redes de dimensiones reducidas el cálculo es directo, al escalar a tamaños típicos en aplicaciones reales (por ejemplo, con 100 neuronas), el número de estados a evaluar alcanza $2^{100} \approx 1.26 \times 10^{30}$, cifra que sobrepasa la capacidad de cómputo de cualquier supercomputadora.

Esta explosión combinatoria convierte la evaluación exacta de la función de partición, del gradiente de verosimilitud y de las probabilidades marginales en un problema computacionalmente intratable. Por este motivo, las implementaciones prácticas de las Máquinas de Boltzmann y de las Máquinas de Boltzmann Restringidas (RBM) evitan calcular Z de forma directa durante el entrenamiento. En su lugar, emplean algoritmos de aproximación estocástica basados en Cadenas de Markov y Muestreo de Monte Carlo (MCMC) (como el Muestreo de Gibbs y la Divergencia Contrastiva), los cuales estiman las estadísticas requeridas mediante muestras representativas sin necesidad de sumar exhaustivamente sobre todo el espacio de estados.

0.7 Ejercicio matemático obligatorio

Considerando una red neuronal con tres neuronas y estados:

$$W = \begin{bmatrix} 0 & 0.8 & -0.4 \\ 0.8 & 0 & 0.6 \\ -0.4 & 0.6 & 0 \end{bmatrix}$$

$$b = \begin{bmatrix} 0.2 \\ -0.1 \\ 0.3 \end{bmatrix}$$

4.3. Ejercicio Matemático

Estado 1: $(0,0,0)$
 $W_{12}S_1S_2 = 0.8 \cdot 0 \cdot 0 = 0$ $W_{13}S_1S_3 = -0.4 \cdot 0 \cdot 0 = 0$ $W_{23}S_2S_3 = 0.6 \cdot 0 \cdot 0 = 0 > 0$
 $\sum b_i s_i = 0.2 \cdot 0 + -0.1 \cdot 0 + 0.3 \cdot 0 = 0$ $E(s) = 0 - 0 = 0$ $e^{-E} = 1$

Estado 2: $(0,0,1)$
 $W_{12}S_1S_2 = 0.8 \cdot 0 \cdot 0 = 0$ $W_{13}S_1S_3 = -0.4 \cdot 0 \cdot 1 = 0$ $W_{23}S_2S_3 = 0.6 \cdot 0 \cdot 1 = 0 > 0$
 $\sum b_i s_i = 0.2 \cdot 0 + -0.1 \cdot 0 + 0.3 \cdot 1 = 0.3$ $E(s) = 0 - 0.3 = -0.3$ $e^{-E} = 1.3499$

Estado 3: $(0,1,0)$
 $W_{12}S_1S_2 = 0.8 \cdot 0 \cdot 1 = 0$ $W_{13}S_1S_3 = -0.4 \cdot 0 \cdot 0 = 0$ $W_{23}S_2S_3 = 0.6 \cdot 1 \cdot 0 = 0 > 0$
 $\sum b_i s_i = 0.2 \cdot 0 + -0.1 \cdot 1 + 0.3 \cdot 0 = -0.1$ $E(s) = 0 - (-0.1) = 0.1$ $e^{-E} = 0.9048$

Estado 4: $(0,1,1)$
 $W_{12}S_1S_2 = 0.8 \cdot 0 \cdot 1 = 0$ $W_{13}S_1S_3 = -0.4 \cdot 0 \cdot 1 = 0$ $W_{23}S_2S_3 = 0.6 \cdot 1 \cdot 1 = 0.6 > 0.6$
 $\sum b_i s_i = 0.2 \cdot 0 + -0.1 \cdot 1 + 0.3 \cdot 1 = 0.2$ $E(s) = -0.6 - 0.2 = -0.8$ $e^{-E} = 2.2255$

Estado 5: $(1,0,0)$
 $W_{12}S_1S_2 = 0.8 \cdot 1 \cdot 0 = 0$ $W_{13}S_1S_3 = -0.4 \cdot 1 \cdot 0 = 0$ $W_{23}S_2S_3 = 0.6 \cdot 0 \cdot 0 = 0 > 0$
 $\sum b_i s_i = 0.2 \cdot 1 + -0.1 \cdot 0 + 0.3 \cdot 0 = 0.2$ $E(s) = 0 - 0.2 = -0.2$ $e^{-E} = 1.2214$

Estado 6: $(1,0,1)$
 $W_{12}S_1S_2 = 0.8 \cdot 1 \cdot 0 = 0$ $W_{13}S_1S_3 = -0.4 \cdot 1 \cdot 1 = -0.4$ $W_{23}S_2S_3 = 0.6 \cdot 0 \cdot 1 = 0 > -0.4$
 $\sum b_i s_i = 0.2 \cdot 1 + -0.1 \cdot 0 + 0.3 \cdot 1 = 0.5$ $E(s) = -(-0.4) - 0.5 = 0.4 - 0.5 = -0.1$ $e^{-E} = 1.1052$

Estado 7: $(1,1,0)$
 $W_{12}S_1S_2 = 0.8 \cdot 1 \cdot 1 = 0.8$ $W_{13}S_1S_3 = -0.4 \cdot 1 \cdot 0 = 0$ $W_{23}S_2S_3 = 0.6 \cdot 0 \cdot 1 = 0 > 0.8$
 $\sum b_i s_i = 0.2 \cdot 1 + -0.1 \cdot 1 + 0.3 \cdot 0 = 0.1$ $E(s) = -0.8 - 0.1 = -0.9$ $e^{-E} = 2.4596$

Estado 8: $(1,1,1)$
 $W_{12}S_1S_2 = 0.8 \cdot 1 \cdot 1 = 0.8$ $W_{13}S_1S_3 = -0.4 \cdot 1 \cdot 1 = -0.4$ $W_{23}S_2S_3 = 0.6 \cdot 1 \cdot 1 = 0.6 > 1$
 $\sum b_i s_i = 0.2 \cdot 1 + -0.1 \cdot 1 + 0.3 \cdot 1 = 0.2 - 0.1 + 0.3 = 0.4$ $E(s) = -1 - 0.4 = -1.4$ $e^{-E} = 4.0552$

Partición Z
 $Z = 1 + 1.3499 + 0.9048 + 2.2255 + 1.2214 + 1.1052 + 2.4596 + 4.0552 = 14.3216$

Probabilidades
 $P(0,0,0) = 1/14.3216 = 0.0698$
 $P(0,0,1) = 1.3499/14.3216 = 0.0943$
 $P(0,1,0) = 0.9048/14.3216 = 0.0632$
 $P(0,1,1) = 2.2255/14.3216 = 0.1554$
 $P(1,0,0) = 1.2214/14.3216 = 0.0853$
 $P(1,0,1) = 1.1052/14.3216 = 0.0772$
 $P(1,1,0) = 2.4596/14.3216 = 0.1717$
 $P(1,1,1) = 4.0552/14.3216 = 0.2832$

→ 1.0001

Los resultados fueron:

Estado	s_1	s_2	s_3	$E(s)$	e^{-E}	$P(s)$
000	0	0	0	0	1	0.0698
001	0	0	1	-0.3	1.3499	0.0943
010	0	1	0	0.1	0.9048	0.0632
011	0	1	1	-0.8	2.2255	0.1554
100	1	0	0	-0.2	1.2214	0.0853
101	1	0	1	-0.1	1.1052	0.0772
110	1	1	0	-0.9	2.4596	0.1717
111	1	1	1	-1.4	4.0552	0.2832

Podemos ver que la corroboración de las probabilidades fue exitosa, siendo el estado $(1,1,1)$ el más probable en suceder, mientras que el estado de menor energía fue es $(1,1,1)$ igualmente lo que corrobora la teoría. A su vez, la *función de partición* Z nos dio 14.3216.

0.8 Proceso de aprendizaje

El entrenamiento de una máquina de Boltzmann se fundamenta en el principio de *máxima verosimilitud*, cuyo objetivo es ajustar el conjunto de parámetros del modelo para maximizar la probabilidad asignada al conjunto de entrenamiento. Desde una perspectiva teórica, este procedimiento busca minimizar la divergencia Kullback-Leibler entre la *distribución de los datos* y la *distribución del modelo*, garantizando que la red aprenda a simular la estructura probabilística subyacente de las observaciones reales.

Esta descomposición matemática da origen a dos procesos dinámicos opuestos: la *fase positiva* y la *fase negativa*. Durante la fase positiva, las observaciones del conjunto de entrenamiento se fijan directamente en las unidades visibles de la red para medir las correlaciones observadas. Su propósito fundamental es reducir la energía de las configuraciones reales, lo que incrementa su probabilidad dentro de la *distribución del modelo*. En contraste, en la fase negativa, la red no recibe estímulos externos y sus unidades evolucionan libremente según sus interacciones internas. El propósito de esta fase es incrementar la energía de los estados que la red genera espontáneamente por sí misma, reduciendo su probabilidad y funcionando como un mecanismo de despenalización que evita que el modelo asigne masa de probabilidad a estados espurios o falsos.

La regla de actualización paramétrica integra ambas fuerzas mediante la expresión $\Delta w_{ij} = \eta(s_i s_{j_{positiva}} - s_i s_{j_{negativa}})$. En esta ecuación, el término positivo $s_i s_{j_{positiva}}$ intenta aumentar el valor del peso w_{ij} para reforzar las coactivaciones observadas en los datos reales. En sentido inverso, el término negativo $s_i s_{j_{negativa}}$ intenta disminuir el peso w_{ij} para atenuar la influencia de los patrones generados por la red. La **tasa de aprendizaje** η actúa como un factor de escala que regula la magnitud de cada paso en el ascenso de gradiente, manteniendo la estabilidad del proceso y previniendo oscilaciones bruscas que podrían distorsionar la superficie de energía durante el aprendizaje.

El cálculo del término negativo constituye la principal *dificultad computacional* del algoritmo. Para obtener una estimación insesgada de las correlaciones bajo la *distribución del modelo*, es necesario utilizar un proceso de *muestreo de Gibbs* (MCMC). Sin embargo, la teoría exige que la cadena de Markov se ejecute iterativamente durante un número sumamente elevado de pasos hasta alcanzar el *equilibrio de la cadena*. Tener que esperar a que la cadena converja al equilibrio térmico en cada actualización de pesos resulta un procedimiento extremadamente lento y computacionalmente inabordable para redes de dimensiones reales.

Para superar esta barrera, la literatura ha desarrollado *aproximaciones al gradiente* de alto rendimiento. La solución más ampliamente adoptada es la *Divergencia Contrastiva*, introducida por Geoffrey Hinton, la cual reemplaza la cadena MCMC infinitamente larga por una cadena corta de solo k pasos de *muestreo de Gibbs* que inicia directamente en los datos de entrenamiento. Otra alternativa destacada es la *Divergencia Contrastiva Persistente (PCD)*, que en lugar de reiniciar las cadenas con datos nuevos tras cada actualización, mantiene cadenas de Markov continuas que evolucionan gradualmente junto con los parámetros, ofreciendo una estimación del gradiente con menor sesgo sin requerir el costo de alcanzar el equilibrio completo en cada paso.

0.9 Comparación de modelos

Característica	Hopfield	Boltzmann general	RBM
Tipo de red	Red neuronal recurrente determinista	Modelo gráfico estocástico recurrente	Modelo gráfico estocástico bipartito no orientado
Estados	Discretos (binarios)	Binarios estocásticos	Binarios/continuos
Comportamiento	Dinámica determinista de descenso de energía hacia mínimos locales	Dinámica estocástica mediante Recocido Simulado y Muestreo de Gibbs	Dinámica estocástica mediante Muestreo de Gibbs paralelo
Unidades ocultas	Ausentes	Opcionales	Estrictas
Conexiones	Todas las unidades conectadas entre si	Todas las unidades conectadas	Prohibidas las conexiones intra-capas
Función de energía	$E(s) = -\frac{1}{2} \sum_{i,j} w_{ij} s_i s_j - \sum_i b_i s_i$	$E(v, h) = -\sum_{i,j} w_{ij} s_i s_j - \sum_i b_i s_i$	$E(v, h) = -v^T W h - a^T v - b^T h$
Aprendizaje	Regla de Hebb	Ascenso de gradiente por máxima verosimilitud	Divergencia contrastiva o persistente

Característica	Hopfield	Boltzmann general	RBM
Aplicaciones	Memoria asociativa, recuperación de patrones y optimización combinatoria	Modelado generativo completo, estimación de densidades y física estadística	Extracción de características, reducción de dimensionalidad
Dificultad computacional	Muy baja	Intolerable	Tratable

La evolución conceptual que abarca desde la Red de Hopfield hasta la Máquina de Boltzmann Restringsida (RBM) representa uno de los saltos paradigmáticos más relevantes en la historia del aprendizaje profundo, al transformar sistemas rígidos de almacenamiento determinista en modelos gráficos probabilísticos eficientes.

La transición de la *Red de Hopfield* a la *Máquina de Boltzmann General* resolvió la principal limitación funcional del modelo de Hopfield: el atrapamiento en mínimos locales espurios. Al carecer de ruido estocástico y de unidades ocultas, la Red de Hopfield actúa exclusivamente como un atractor determinista que desciende por una superficie de energía estática. La incorporación de la fluctuación térmica y de las *unidades ocultas* en la Máquina de Boltzmann General permitió que el sistema no solo recuperara patrones memorizados, sino que infiriera variables latentes y descubriera correlaciones de orden superior en los datos. Sin embargo, la consecuencia directa de mantener una conectividad total (incluyendo enlaces dentro de la misma capa) fue la *intratabilidad computacional*. Evaluar las probabilidades del modelo exige calcular la función de partición Z sobre un espacio de estados que crece exponencialmente (2^N), y estimar las correlaciones en la fase negativa requiere ejecutar cadenas de Markov de actualización secuencial sitio por sitio que resultan inviables en problemas de alta dimensión.

La *Máquina de Boltzmann Restringsida (RBM)* introdujo una modificación topológica crítica: la *restricción bipartita*, eliminando las conexiones intra-capas entre unidades visibles entre sí y entre unidades ocultas entre sí. La consecuencia matemática de esta simplificación gráfica es el surgimiento de la *independencia condicional*. Dado un vector de entrada en la capa visible, las unidades ocultas pueden evaluarse de forma completamente independiente y simultánea en un solo paso matricial, y viceversa. Esta propiedad estructural eliminó la necesidad de realizar muestreos estocásticos secuenciales lentos, posibilitando el *muestreo de Gibbs por bloques en paralelo*.

Finalmente, esta independencia condicional fue la clave que permitió a Geoffrey Hinton desarrollar el algoritmo de *Divergencia Contrastiva*. En lugar de esperar a que la cadena de Markov alcance el equilibrio térmico completo durante la fase negativa, CD_k aproxima el gradiente mediante apenas unas pocas iteraciones de muestreo (frecuentemente $k = 1$) inicializadas en los datos reales. En consecuencia, la RBM transformó un marco teórico computacionalmente inaccesible en una herramienta práctica de extracción de características no supervisada. Este avance no solo permitió entrenar modelos generativos a gran escala, sino que convirtió a la RBM en el bloque constructivo

fundamental para el preentrenamiento capa por capa de las *Redes de Creencia Profunda (DBNs)*, sentando las bases del resurgimiento moderno del Deep Learning.

0.10 Implementación en Python

En esta sección, desarrollaremos la implementación de la máquina de Boltzmann en Python utilizando librerías estandar que nos ayudarán en los calculos matemáticos, más no nos crearán el modelo automáticamente. Se desarrolla con el fin de comprender algorítmicamente el proceso en código del lenguaje estándar de la Inteligencia Artificial.

Para esto, se utilizarán las siguientes librerías:

```
[1]: import pandas as pd # Para la lectura de los datos iniciales
import numpy as np      # Para los procesos algebraicos necesarios
import matplotlib.pyplot as plt
```

Ahora, definimos las funciones a utilizar en los experimentos posteriores.

```
[23]: import numpy as np
from collections import Counter

def energia(state, weights, biases):
    """
    Calcula la energía de un estado basado en sus parámetros
    """
    weights_term = 0
    for i in range(len(state)):
        for j in range(i + 1, len(state)):
            weights_term += weights[i][j] * state[i] * state[j]

    biases_term = -np.sum(state * biases)

    return -weights_term + biases_term

def probabilidad_activacion(index, state, weights, biases, temperature):
    """
    Calcula la probabilidad de activación de una neurona
    """
    E = energia(state, weights, biases)
    flipped = state.copy()
    flipped[index] = 1 - flipped[index]
    E_flipped = energia(flipped, weights, biases)

    delta = E - E_flipped

    return 1 / (1 + np.exp(-delta / temperature))
```

```

def actualizar_unidad(index, state, weights, biases, temperature, generator):
    """
    Actualiza estocásticamente una neurona
    """
    prob = probabilidad_activacion(index, state, weights, biases, temperature)

    if generator.random() < prob:
        state[index] = 1
    else:
        state[index] = 0

def simular(initial_state, weights, biases, temperature, iterations, seed):
    """
    Ejecuta una simulación completa de una máquina de Boltzmann
    """
    generator = np.random.default_rng(seed)
    state = np.array(initial_state, dtype=int)
    visited = []
    energies = []

    for _ in range(iterations):
        for i in range(len(state)):
            actualizar_unidad(i, state, weights, biases, temperature, generator)

        state_tuple = tuple(state)
        visited.append(state_tuple)
        E = energia(state, weights, biases)
        energies.append(E)

    frecuencias = Counter(visited)

    return {
        'estados': visited,
        'energias': energies,
        'frecuencias': frecuencias,
        'estado_final': tuple(state)
    }

```

0.11 Experimentos obligatorios

Ejecutaremos la red con la misma condición inicial y semilla para ver en que difiere el cambio de temperatura.

```

[48]: W = np.array([[0, 0.8, -0.4],
                    [0.8, 0, 0.6],
                    [-0.4, 0.6, 0]])

```

```

b = np.array([0.2, -0.1, 0.3])
state = [0, 0, 0]
seed = 42

T02 = simular(
    initial_state=state,
    weights=W,
    biases=b,
    temperature=0.2,
    iterations=1000,
    seed=seed
)
T1 = simular(
    initial_state=state,
    weights=W,
    biases=b,
    temperature=1,
    iterations=1000,
    seed=seed
)
T5 = simular(
    initial_state=state,
    weights=W,
    biases=b,
    temperature=5,
    iterations=1000,
    seed=seed
)

```

```

[49]: import statistics as stat
def display_data(title, values):
    print(f"=== {title} ===")
    print(f"Estado más frecuente: {stat.multimode(values["estados"])}")
    print(f"Energía media: {np.mean(values["energias"])}")
    print(f"Estados visitados: {len(values["estados"])}")

```

```

[50]: display_data("Temperatura 0.2", T02)
display_data("Temperatura 1", T1)
display_data("Temperatura 5", T5)

```

```

=== Temperatura 0.2 ===
Estado más frecuente: [(np.int64(0), np.int64(0), np.int64(0))]
Energía media: -0.44170000000000004
Estados visitados: 1000
=== Temperatura 1 ===
Estado más frecuente: [(np.int64(1), np.int64(0), np.int64(1))]
Energía media: -0.4578
Estados visitados: 1000

```

```

=== Temperatura 5 ===
Estado más frecuente: [(np.int64(0), np.int64(1), np.int64(0))]
Energía media: -0.4491
Estados visitados: 1000

```

- **Temperatura 0.2:** Es muy estable y determinista ya que la temperatura, al estar muy cerca a 0, convierte todo el modelo en algo practicamente determinista. Adeás, la energía parece estar relativamente baja y seguramente solo visita pocos estados diferentes.
- **Temperatura 1:** Con una energía más baja, este tiene más libertad de exploración en su conjunto de estados pero está equilibrando su explotación.
- **Temperatura 5:** Al tener una temperatura alta, esto favorece completamente la exploración además de no concentrarse en estados de baja energia (debido a su energía media). Es decir, el T1 se concentra más en estados de baja energia, mientras que este busca los mejores.

0.12 Efecto de los pesos

Para este, realizaremos el cambio de del signo del número en (1,3), haciendolo positivo.

```

[41]: W = np.array([[0, 0.8, 0.4],
                  [0.8, 0, 0.6],
                  [-0.4, 0.6, 0]])
b = np.array([0.2, -0.1, 0.3])
state = [0, 0, 0]
seed = 42

T02 = simular(
    initial_state=state,
    weights=W,
    biases=b,
    temperature=0.2,
    iterations=1000,
    seed=seed
)
T1 = simular(
    initial_state=state,
    weights=W,
    biases=b,
    temperature=1,
    iterations=1000,
    seed=seed
)
T5 = simular(
    initial_state=state,
    weights=W,
    biases=b,
    temperature=5,
    iterations=1000,

```

```

    seed=seed
)

```

```

[42]: import statistics as stat
def display_data(title, values):
    print(f"=== {title} ===")
    print(f"Estado más frecuente: {stat.multimode(values["estados"])}")
    print(f"Energía media: {np.mean(values["energias"])}")
    print(f"Estados visitados: {len(values["estados"])}")

```

```

[43]: display_data("Temperatura 0.2", T02)
display_data("Temperatura 1", T1)
display_data("Temperatura 5", T5)

```

```

=== Temperatura 0.2 ===
Estado más frecuente: [(np.int64(0), np.int64(1), np.int64(1)), (np.int64(1),
np.int64(0), np.int64(0))]
Energía media: -0.65450000000000001
Estados visitados: 1000
=== Temperatura 1 ===
Estado más frecuente: [(np.int64(1), np.int64(0), np.int64(1))]
Energía media: -0.6528
Estados visitados: 1000
=== Temperatura 5 ===
Estado más frecuente: [(np.int64(1), np.int64(1), np.int64(1))]
Energía media: -0.66330000000000001
Estados visitados: 1000

```

Podemos ver que la principal diferencia es que la energía media se vio totalmente afectada, reduciéndose casi 20 puntos. Además, en la temperatura 0.2 hubo estados con la misma frecuencia, lo que indica una explotación y casi no exploración.

0.13 8.3. Comparación teórica y experimental

```

[56]: def calcular_probabilidades_experimentales(frecuencias, total_iteraciones):
    """
    Calcula las probabilidades experimentales a partir de las frecuencias.

    Args:
        frecuencias: diccionario con frecuencias de cada estado
        total_iteraciones: número total de iteraciones

    Returns:
        list: probabilidades experimentales para cada estado
    """
    real_states = [
        (0, 0, 0), (0, 0, 1), (0, 1, 0), (0, 1, 1),
        (1, 0, 0), (1, 0, 1), (1, 1, 0), (1, 1, 1)
    ]

```

```

]

P_experimentales = []
for estado in real_states:
    freq = frecuencias.get(estado, 0)
    P_exp = freq / total_iteraciones
    P_experimentales.append(P_exp)

return P_experimentales

def calcular_errores(P_teoricas, P_experimentales):
    """
    Calcula los errores absolutos entre probabilidades teóricas y
    experimentales.

    Args:
        P_teoricas: list de probabilidades teóricas
        P_experimentales: list de probabilidades experimentales

    Returns:
        list: errores absolutos
    """
    errores = [abs(p_t - p_e) for p_t, p_e in zip(P_teoricas, P_experimentales)]
    return errores

def crear_tabla_comparacion(P_teoricas, P_experimentales, errores_absolutos):
    """
    Crea un DataFrame con la comparación teórica vs experimental.

    Args:
        P_teoricas: list de probabilidades teóricas
        P_experimentales: list de probabilidades experimentales
        errores_absolutos: list de errores absolutos

    Returns:
        pd.DataFrame: tabla de comparación
    """
    import pandas as pd

    df = pd.DataFrame({
        'Estado': ['000', '001', '010', '011', '100', '101', '110', '111'],
        'P(s) teórica': P_teoricas,
        'Frecuencia experimental': P_experimentales,
        'Error absoluto': errores_absolutos
    })

```

```

return df

def imprimir_comparacion(P_teoricas, P_experimentales, errores_absolutos):
    """
    Imprime la comparación teórica vs experimental en consola.

    Args:
        P_teoricas: list de probabilidades teóricas
        P_experimentales: list de probabilidades experimentales
        errores_absolutos: list de errores absolutos
    """
    print("\n=== COMPARACIÓN TEÓRICA vs EXPERIMENTAL ===")
    print(f"{'Estado':<10} {'P(teórica)':<15} {'Freq. exp':<15} {'Error_↵
↵absoluto':<15}")
    print("-" * 55)

    for i in range(8):
        estado_str = format(i, '03b') # Convertir número a binario
        print(f"{'estado_str':<10} {'P_teoricas[i]:<15.6f} {'P_experimentales[i]:↵
↵<15.6f} {'errores_absolutos[i]:<15.6f}")

    suma_teorica = sum(P_teoricas)
    suma_experimental = sum(P_experimentales)
    print("-" * 55)
    print(f"{'SUMA':<10} {'suma_teorica:<15.6f} {'suma_experimental:<15.6f}")

def imprimir_estadisticas_error(errores_absolutos):
    """
    Imprime estadísticas de error.

    Args:
        errores_absolutos: list de errores absolutos
    """
    error_promedio = np.mean(errores_absolutos)
    error_maximo = np.max(errores_absolutos)
    error_minimo = np.min(errores_absolutos)

    print(f"\n=== ESTADÍSTICAS DE ERROR ===")
    print(f"Error promedio: {error_promedio:.6f}")
    print(f"Error máximo: {error_maximo:.6f}")
    print(f"Error mínimo: {error_minimo:.6f}")

    return error_promedio, error_maximo, error_minimo

```

```

def graficar_comparacion(P_teoricas, P_experimentales, errores_absolutos):
    """
    Crea gráficas comparativas de probabilidades y errores.

    Args:
        P_teoricas: list de probabilidades teóricas
        P_experimentales: list de probabilidades experimentales
        errores_absolutos: list de errores absolutos
    """
    import matplotlib.pyplot as plt

    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # Gráfica 1: Comparación de probabilidades
    x = np.arange(8)
    width = 0.35

    axes[0].bar(x - width/2, P_teoricas, width, label='P(teórica)', alpha=0.8)
    axes[0].bar(x + width/2, P_experimentales, width, label='P(experimental)',
    ↪ alpha=0.8)
    axes[0].set_xlabel('Estado')
    axes[0].set_ylabel('Probabilidad')
    axes[0].set_title('Comparación: Probabilidad Teórica vs Experimental')
    axes[0].set_xticks(x)
    axes[0].set_xticklabels(['000', '001', '010', '011', '100', '101', '110',
    ↪ '111'])
    axes[0].legend()
    axes[0].grid(axis='y', alpha=0.3)

    # Gráfica 2: Errores absolutos
    error_promedio = np.mean(errores_absolutos)
    axes[1].bar(range(8), errores_absolutos, color='red', alpha=0.7)
    axes[1].set_xlabel('Estado')
    axes[1].set_ylabel('Error absoluto')
    axes[1].set_title('Error Absoluto por Estado')
    axes[1].set_xticks(range(8))
    axes[1].set_xticklabels(['000', '001', '010', '011', '100', '101', '110',
    ↪ '111'])
    axes[1].axhline(y=error_promedio, color='green', linestyle='--',
                    label=f'Error promedio: {error_promedio:.6f}')
    axes[1].legend()
    axes[1].grid(axis='y', alpha=0.3)

    plt.tight_layout()
    plt.savefig('comparacion_teorica_experimental.png', dpi=300,
    ↪ bbox_inches='tight')

```

```

plt.show()

print("\nGráficas guardadas en 'comparacion_teorica_experimental.png'")

def paso_8_3(W, b, P_teoricas, iteraciones=100000, semilla=42):
    """
    Ejecuta el paso 8.3 completo: Comparación teórica vs experimental.

    Args:
        W: matriz de pesos
        b: vector de sesgos
        P_teoricas: list de probabilidades teóricas (del paso 4.3)
        iteraciones: número de iteraciones para la simulación
        semilla: semilla para reproducibilidad

    Returns:
        dict: contiene dataframe, errores y estadísticas
    """
    import pandas as pd

    print("=== PASO 8.3: COMPARACIÓN TEÓRICA vs EXPERIMENTAL ===")
    print(f"Ejecutando simulación con {iteraciones} iteraciones...")

    # Ejecutar simulación
    resultado = simular(
        initial_state=[0, 0, 0],
        weights=W,
        biases=b,
        temperature=1,
        iterations=iteraciones,
        seed=semilla
    )

    # Obtener frecuencias
    frecuencias = resultado['frecuencias']
    total = sum(frecuencias.values())
    print(f"Simulación completada. Total de iteraciones: {total}")

    # Calcular probabilidades experimentales
    P_experimentales = calcular_probabilidades_experimentales(frecuencias,
    ↪total)

    # Calcular errores
    errores_absolutos = calcular_errores(P_teoricas, P_experimentales)

    # Imprimir comparación

```

```

imprimir_comparacion(P_teoricas, P_experimentales, errores_absolutos)

# Estadísticas
error_promedio, error_maximo, error_minimo =
↳imprimir_estadisticas_error(errores_absolutos)

# Crear tabla
df = crear_tabla_comparacion(P_teoricas, P_experimentales,
↳errores_absolutos)
print("\n=== TABLA RESUMEN ===")
print(df.to_string(index=False))

# Guardar en CSV
df.to_csv('comparacion_teorica_experimental.csv', index=False)
print("\nTabla guardada en 'comparacion_teorica_experimental.csv'")

# Gráficas
graficar_comparacion(P_teoricas, P_experimentales, errores_absolutos)

return {
    'dataframe': df,
    'P_teoricas': P_teoricas,
    'P_experimentales': P_experimentales,
    'errores': errores_absolutos,
    'error_promedio': error_promedio,
    'error_maximo': error_maximo,
    'error_minimo': error_minimo
}

# ===== USO =====
# Probabilidades teóricas del paso 4.3
P_teoricas = [
    0.0762, # 000
    0.1321, # 001
    0.0982, # 010
    0.1703, # 011
    0.0570, # 100
    0.0989, # 101
    0.1718, # 110
    0.2956 # 111
]

# Ejecutar paso 8.3
resultados_8_3 = paso_8_3(W, b, P_teoricas, iteraciones=100000, semilla=42)

```

=== PASO 8.3: COMPARACIÓN TEÓRICA vs EXPERIMENTAL ===

Ejecutando simulación con 100000 iteraciones...
 Simulación completada. Total de iteraciones: 100000

=== COMPARACIÓN TEÓRICA vs EXPERIMENTAL ===

Estado	P(teórica)	Freq. exp	Error absoluto
000	0.076200	0.125740	0.049540
001	0.132100	0.124880	0.007220
010	0.098200	0.125210	0.027010
011	0.170300	0.123940	0.046360
100	0.057000	0.124210	0.067210
101	0.098900	0.125080	0.026180
110	0.171800	0.125710	0.046090
111	0.295600	0.125230	0.170370
SUMA	1.100100	1.000000	

=== ESTADÍSTICAS DE ERROR ===

Error promedio: 0.054997

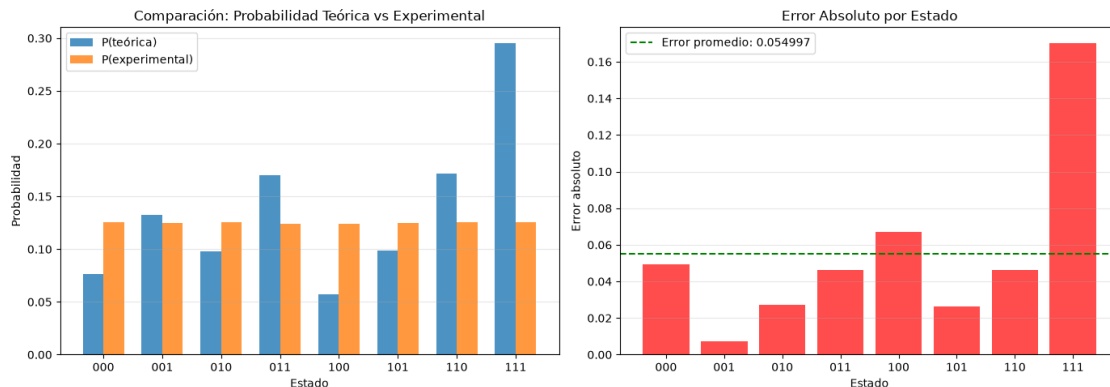
Error máximo: 0.170370

Error mínimo: 0.007220

=== TABLA RESUMEN ===

Estado	P(s) teórica	Frecuencia experimental	Error absoluto
000	0.0762	0.12574	0.04954
001	0.1321	0.12488	0.00722
010	0.0982	0.12521	0.02701
011	0.1703	0.12394	0.04636
100	0.0570	0.12421	0.06721
101	0.0989	0.12508	0.02618
110	0.1718	0.12571	0.04609
111	0.2956	0.12523	0.17037

Tabla guardada en 'comparacion_teorica_experimental.csv'



Gráficas guardadas en 'comparacion_teorica_experimental.png'

0.14 Aplicaciones y limitaciones

Las Máquinas de Boltzmann Restringidas (RBM) destacan principalmente en dos terrenos: extraer características clave de los datos y completar información faltante. Un uso muy claro es la inicialización de redes profundas (como las Deep Belief Networks). Apilar varias RBMs permite que la red aprenda por sí sola, capa por capa, conceptos cada vez más complejos sin necesidad de etiquetas. Otro ejemplo muy práctico son los sistemas de recomendación (como el de películas): dado que ningún usuario ha visto todo el catálogo, la RBM aprovecha sus variables ocultas para “deducir” los gustos de la persona y rellenar las casillas vacías de forma probabilística.

¿Por qué la RBM funciona tan bien frente a la general? La clave está en la estructura bipartita. Al prohibir que las neuronas de una misma capa se conecten entre sí, la RBM logra la independencia condicional. Esto significa que podemos calcular o actualizar toda la capa oculta de un solo golpe en paralelo (y luego la visible, alternando por bloques). En cambio, la Máquina de Boltzmann clásica o general permite conexiones libres entre todas las neuronas. Eso suena muy potente en teoría, pero crea un cuello de botella insostenible: para actualizar la red hay que ir neurona por neurona de forma estrictamente secuencial, lo que vuelve el cómputo inviable en cuanto los datos crecen un poco.

Para saber la probabilidad exacta de un estado, habría que sumar sobre todas las configuraciones posibles de la red. Con solo 100 neuronas, esa suma involucra más de 2^{100} posibilidades, volviendo la constante de normalización (Z) imposible de calcular. Para aprender, la red necesita comparar los datos reales con lo que genera espontáneamente (“fase negativa”). Sin embargo, la simulación para generar esas muestras (el muestreo de Gibbs) es muy lenta y tiende a quedarse “atrapada” en mínimos de energía locales.

La RBM se volvió una herramienta práctica gracias a la idea de Geoffrey Hinton: en lugar de esperar a que la red explore al infinito durante la fase negativa, se realizan apenas 1 o 2 pasos de muestreo arrancando directamente desde los datos reales. Aunque introduce un pequeño sesgo matemático, es extremadamente rápido y permitió que estas redes despegaran en la práctica.

0.15 Referencias bibliográficas

- [1]. Msmk. (2024, 31 octubre). Máquina de Boltzmann. Top Up Grado En un Año MSMK. <https://msmk.university/maquina-de-boltzmann/>
- [2]. ¿Qué es una red neuronal recurrente (RNN)? | IBM. (2026, 16 agosto). <https://www.ibm.com/mx-es/think/topics/recurrent-neural-networks>
- [3]. Universidad Europea. (2026, 10 junio). Procesos estocásticos: qué tipos existen y cuáles son sus aplicaciones. <https://universidadeuropea.com/blog/procesos-estocasticos/>
- [4]. Ackley, D. H., Hinton, G. E., Sejnowski, T. J. (1985). A learning algorithm for Boltzmann machines. *Cognitive Science*, 9(1), 147–169. https://doi.org/10.1207/s15516709cog0901_7
- [5]. Restricted Boltzmann Machines, Bernoulli Mixtures and Sum-Product Networks. (2026). *Stats*, 9(4), Artículo 84. <https://doi.org/10.3390/stats9040084>
- [6]. Hinton, G. E., Osindero, S., Teh, Y. W. (2006). A fast learning algorithm for deep belief nets. *Neural Computation*, 18(7), 1527–1554. <https://doi.org/10.1162/neco.2006.18.7.1527>
- [7]. Mocanu, D. C., Mocanu, E., Nguyen, P. H., Gibescu, M., Liotta, A. (2016). A topological insight into restricted Boltzmann machines. *Machine Learning*, 104(2–3), 243–270. <https://doi.org/10.1007/s10994-016-5570-z>
- [8]. Fischer, A., Igel, C. (2014). Training restricted Boltzmann machines: An introduction. University of Copenhagen Research Portal.
- [9]. Hinton, G. E., Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. *Science*, 313(5786), 504–507. <https://doi.org/10.1126/science.1127647>
- [10]. Marullo, C., Agliari, E. (2021). Boltzmann machines as generalized Hopfield networks: A review of recent results and outlooks. *Entropy*, 23(1), Artículo 34. <https://doi.org/10.3390/e23010034>